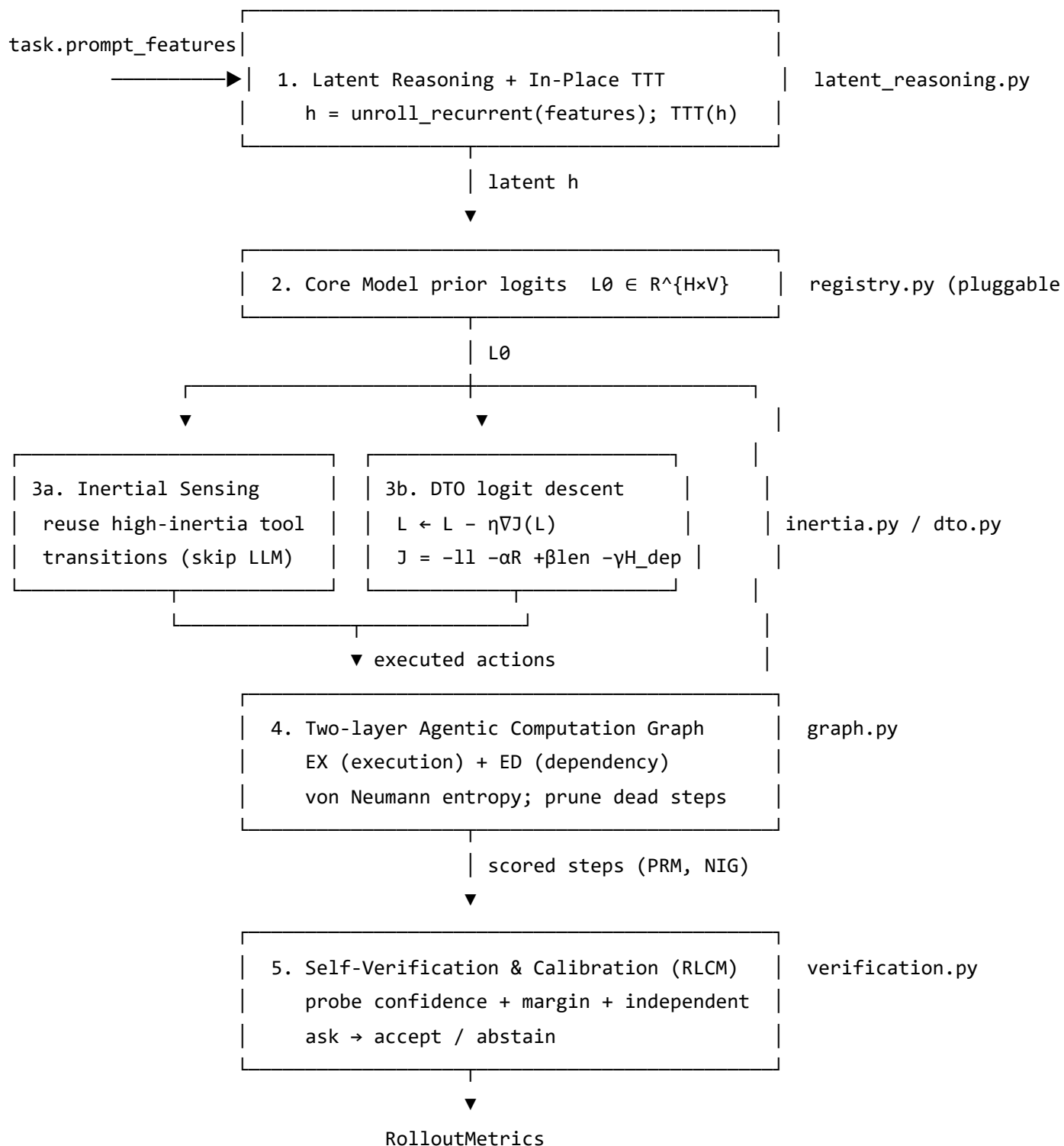


# ACDAN Architecture

ACDAN unifies latent reasoning, first-order differentiable optimisation, dynamic graph control, and confidence calibration into a single test-time agent. This document explains the data flow and the responsibility of each module.

# End-to-end data flow



# Module responsibilities

## 1. Latent Reasoning + In-Place TTT ( `latent_reasoning.py` )

Deepens reasoning by unrolling a recurrent block in feature space (no KV-cache growth). In-place TTT adapts a small input projection per task via a self-supervised reconstruction loss. Output: a task-conditioned latent whose *quality* gates the sharpness of downstream signals.

## 2. Core Model ( `registry.py` )

Emits the prior action-logit matrix  $L_0$  ( $H$  steps  $\times$   $V$  actions). Mock by default; pluggable to a real LLM action head.

## 3. Planning

- **DTO** ( `dto.py` ): the heart of ACDAN. Treats the plan as a continuous logit matrix and runs  $T$  first-order updates that trade off core-model likelihood, PRM process reward, an anti-overthinking length penalty, and a dependency-diversity (von Neumann entropy) term. Gradients are analytic.
- **Inertial Sensing** ( `inertia.py` ): for familiar tool transitions, fills the action directly from a learned Markov model, *skipping the LLM planning call* — the main inference-cost saver.

## 4. Two-layer Graph ( `graph.py` )

Builds the Execution and Dependency layers over the executed plan, measures the dependency layer's von Neumann entropy, and prunes "dead" (redundant) steps — the anti-overthinking / token-reduction mechanism.

## 5. Self-Verification & Calibration ( `verification.py` )

The RLCM confidence probe predicts  $P(\text{correct})$  from the latent; a margin score and an *independent* evidence query are blended into a calibrated confidence used for accept/abstain risk control.

# Design principles

- **Offline-first**: no downloads; everything is seeded numpy.
- **Interface-driven**: real backends drop in at the `registry.py` seam.
- **Ablation-native**: every module is a single boolean in `AblationFlags`.
- **Analytic & testable**: gradients are checked against finite differences.
- **Honest**: mock components are labelled; we do not claim SOTA.

See `math.md` for the formal objective and update rules, and `module_to_paper_mapping.md` for the line-level map.